

Fuzzing Libsixel

A report for CS-412

Maria Bennani
EPFL

Sean Perazzolo
EPFL

Valuthy Karunakaran
EPFL

Introduction

This report details the design, execution, and analysis of a coverage-guided fuzzing campaign on version 1.8.7 of the open-source library `libsixel` using the AFL++ fuzzer.

`libsixel` is an encoder/decoder library for SIXEL, an image format for printer and terminal imaging.

The project is comprised of two parts: white-box fuzzing (source-instrumented) and black-box (binary-only, using QEMU).

1 Harness Design

1.1 Selected Entrypoint and Considered Alternatives

For this campaign, we selected the **encoding path** of the library, specifically targeting the `sixel_encoder_encode` function.

Even though the guidelines suggested fuzzing the decoding path, recently identified vulnerabilities were due to bugs in the encoding path (see CVE-2026-33018). This made the encoder a highly attractive target.

We initially considered fuzzing the high-level `img2sixel` function. However, we rejected this approach because high-level wrappers introduce unnecessary overhead (e.g. command-line argument parsing, I/O operations, ...). By writing a custom harness directly around `sixel_encoder_encode`, we skip all non-encoding related functionality.

1.2 Data Flow and Guards

Each AFL++ test case is split into two parts: an 8-byte control header and the raw image payload.

The first 8 bytes are parsed by the harness and mapped to configuration options. They are passed to the library via

`sixel_encoder_setopt` to configure encoder options such as color mode, quality, diffusion, resize/crop behavior, and animation-related flags.

The remaining bytes represent the image and are written to a temporary file. The latter is passed to `sixel_encoder_encode`, which expects a file path as input. Note that the harness receives each AFL++ test case through `stdin`, rather than through the usual `@@` filename argument, avoiding I/O overhead.

The main data flow is therefore:

```
1 AFL++ testcase -> stdin -> [8-byte option header] + [GIF
  payload] -> temporary GIF file -> sixel_encoder_encode
```

This lets AFL++ mutate both the image contents and the encoder configuration. The option bytes are mapped to fixed valid strings, to ensure the fuzzer explores many combinations of `libsixel` behavior.

The harness contains several guards to allow the fuzzer to efficiently explore the library:

1. A guard ensures that the test case is at least 8 bytes long to contain the control header. This is needed because the harness expects the first 8 bytes to be valid options, and shorter inputs would be rejected before reaching the library code.
2. A guard ensures that the test case also contains a complete 6-byte GIF header after the control bytes. This is needed because the harness reconstructs an image file before calling the library API. Inputs shorter than this would only exercise trivial file rejection and would not reach the interesting decoder or encoder code. *Note that this guard can be removed to allow broader exploration of the file parsing logic with `make FUZZ_BROAD_LOADER=1`.*
3. We then have various other guards to ensure the harness passes valid options / files to the library, such as

```
if (fd < 0) continue; to ensure the file has been
successfully written or if (status != SIXEL_OK)
continue; which ensures the option values are valid
and accepted by the library.
```

We also guard all harness option selection by mapping arbitrary AFL++ bytes into finite option sets using masks or modulo operations. This keeps crashes attributable to the library code and not to the harness (e.g. by preventing invalid option values from being passed to the library).

Finally the resize and crop options are chosen from small fixed dictionaries. This bounds generated image dimensions and crop regions, preventing AFL++ from spending time on huge allocations, very slow transformations, or timeout-heavy inputs while still exercising scaling, cropping, and resampling code.

2 Instrumentation and Sanitizers

2.1 Target Version

We built **libsixel v1.8.7**, the latest stable release of the library. At the time of writing, two release candidates were published (r1, r2), and they address security vulnerabilities in the encoding path: these CVEs act as a benchmark for our campaign as we should be able to find them with our setup, and they provide a good case study, as well as motivate our choice of fuzzing the encoder instead of the decoder.

2.2 Coverage Instrumentation

We instrumented the library with AFL++’s `afl-clang-lto / afl-clang-lto++`, which inserts compile-time edge coverage. This gives AFL++ precise feedback for input selection.

```
1 CC=afl-clang-lto CXX=afl-clang-lto++
```

2.3 AddressSanitizer

We also built the library with AddressSanitizer (ASan) to detect memory safety bugs. This allows us to catch memory corruption issues that might not immediately cause a crash, improving our chances of finding security vulnerabilities. If omitted, AFL++ would still find hard crashes, but many invalid reads/writes would remain silent or become harder-to-reproduce later crashes.

```
1 ENV AFL_USE_ASAN=1
```

Because ASan reserves a large virtual address space, we run AFL++ with `-m none`; otherwise AFL++’s default memory limit can kill valid ASan-instrumented executions.

At runtime, we used the following `ASAN_OPTIONS`

```
1 detect_leaks=0
2 abort_on_error=1
3 symbolize=0
```

Leak detection was disabled because the persistent harness intentionally runs many iterations in one process, and leak reports at process exit are less useful for this campaign than immediate memory-safety crashes. `abort_on_error=1` ensures sanitizer findings terminate the process in a way AFL++ records as a crash. `symbolize=0` avoids the overhead of online symbolization during fuzzing; symbolization can be done later during triage.

2.4 Build Configuration

The flags `-g -O2` were used to include debug symbols for better crash triage and to optimize the code for more realistic performance.

`--prefix=/usr/local` ensures the harness links against the freshly built instrumented library. `--with-gd`, `--with-jpeg`, and `--with-png` enable the image backends relevant to realistic encoder inputs; `--with-gd` is especially important for GIF handling. `--with-libcurl=no`, `--with-gdk-pixbuf2=no`, and `--disable-python` remove network, GUI, and language-binding components that our harness does not exercise.

2.5 Linking

When linking the final harness, we used `-Wl,--whole-archive -lsixel -Wl,--no-whole-archive`. This forces the linker to include the full static `libsixel` archive in the final binary instead of only the object files directly referenced by the harness. This is useful for instrumentation accounting and comparison because the final binary contains the library code rather than only a minimal linker-selected subset. Without `--whole-archive`, the binary would be smaller, but some instrumented library edges would be absent from the final executable.

2.6 Patching

The library was configured with `--disable-shared --enable-static`. This makes the harness link against a static `libsixel` archive instead of a shared library. Static linking makes the fuzzing binary self-contained and ensures that the instrumented library code is the code executed by AFL++. If we used an uninstrumented shared system library by mistake, AFL++ would mainly see coverage from the harness rather than from the target library.

We did not apply a source patch to `libsixel`. In particular, there was no checksum-removal patch comparable to the `libpng` CRC patch discussed in the handout. Our main format was GIF, whose magic-byte check is handled at the harness level by preserving or filtering for GIF87a/GIF89a.

The optional `TRUEVISION_PATCH=1` setting in our setup is not a patch to the target library; it is a harness-side filter that skips very small image payloads to avoid shallow file-detection behavior and focus executions on deeper parsing paths.

3 Seed Corpus and Dictionary

We selected a raw seed corpus of 10 small GIF files that cover a variety of features of the GIF format. The corpus includes both GIF87a and GIF89a files, 1x1 edge-case images, transparent GIFs, interlaced and non-interlaced variants of the same image, small 32x32 thumbnails, and larger 100x100 images.

One seed was selected in particular (`poc1_gif_oob.gif`) because it is a proof-of-concept input for the vulnerability classes we aimed to rediscover, more on this in Section 4.

Before fuzzing, each raw GIF is wrapped with the 8-byte harness control header described earlier, yielding inputs of the form `control header || GIF payload`.

To keep the baseline campaign fast, we excluded expensive seeds from the default corpus. They can optionally be included by using the appropriate flag when running the campaign.

3.1 Mutation Strategy Analysis

The AFL++ strategy-yield table shows that dictionary-based mutations contributed 136 new paths. The dictionary row reports 136/458k, 0/459k, 0/0, 0/0, meaning that the first dictionary mutation mode found 136 interesting inputs over about 458k executions. The havoc/splice row reports 973/1.88M, 0/0, so havoc contributed 973 new paths over about 1.88M executions, while splicing did not contribute in this run; AFL++ did not spend executions in the splice stage.

The numbers are reported in Table 4

3.2 Dictionary

To optimize the mutation process, we provided AFL++ with a GIF dictionary (which can be found at [this link](#)). The fuzzer can therefore directly inject these known tokens (headers, section markers, etc.) instead of having to discover them through random mutations.

4 Campaign Analysis

The main greybox campaign **ran for over an hour**. The AFL++ status screen is shown in Figure 1, and the `afl-plot` graphs are shown in Figure 2, Figure 3, Figure 4, and Figure 5.

At the end of the run, AFL++ reported a **corpus count of 1515 inputs**, with 122 favored items and 242 inputs that discovered new edges.

The campaign completed two full queue cycles (not shown in the status screen). It also found **51 unique crashes** and 58 saved hangs.

AFL++ reported **99.96% stability**, which means that repeated executions of the same inputs almost always produced the same coverage.

The bitmap density was 10.34% for the whole corpus, with 5.44 bits per tuple. This indicates that the campaign reached a meaningful part of the instrumented program, while still remaining far from bitmap saturation. The final execution speed was also lower than in the short benchmark runs, which is expected because the evolved queue contained slower inputs that reached deeper GIF decoding paths and included hang-producing test cases.

The edge curve rises quickly at the beginning of the run and then continues as smaller step increases. This is the expected shape for a useful fuzzing campaign: AFL++ first discovers shallow parsing paths from the initial GIF seeds, then later reaches deeper behavior through mutation and queue cycling.

The status screen reports that a new path had been found 58 seconds before the screenshot, so the campaign was still productive when it was stopped. For this reason, **we do not consider the campaign saturated**. The run is long enough to validate the harness and demonstrate that AFL++ reaches meaningful decoder behavior, especially since it found 51 unique crashes. However, because new paths were still being found near the end of the hour-long run, a longer campaign would likely discover additional coverage and possibly more crashes.

5 Crash Triage

One of the two distinct crashes we found was a **heap-based buffer overflow**, specifically it is an **out-of-bounds write** in `libsixel`'s GIF decoder. The rest of this section analyzes crash `id:000047` from `findings/default/crashes`, which is one of the many inputs that triggers this bug. The input reaches `load_gif()`, which calls

`gif_init_frame()`. AddressSanitizer reports an out-of-bounds write at `fromgif.c:241`:

```
frame->palette[pg->transparent * 3 + 0] = bgcolor[0]
```

The root cause is that the GIF transparency index from the Graphic Control Extension is used as an index into `frame->palette` without checking that it is smaller than the number of palette entries. In the crashing input, the transparent index is `0xdf`, while the GIF is a tiny `1x1` image with a much smaller palette. This causes writes past the heap allocation for `frame->palette`.

Under ASan the program aborts with SIGABRT, without ASan the bug may corrupt adjacent heap memory.

This bug does not have an assigned CVE but has already been reported and fixed in the release candidate: <https://github.com/saitoha/libsixel/issues/220>.

The triage included reproducing the bug with our harness, then with `img2sixel` and finally with a minimized version.

All commands necessary for the full triage, as well as the stack trace, are in the Appendix Section 9.5

6 Attack Surface Analysis

The `libsixel` GitHub repository lists several applications that use the library such as `w3m` (text-based web browser and pager) and `RetroArch` (frontend for video game system emulators).

A concrete attack scenario for `w3m` could be a malicious web page containing a specially crafted GIF image that exploits the heap-based buffer overflow in `libsixel`'s GIF decoder (as described in the previous section). When a user visits the page with `w3m`, the vulnerable code path is triggered, potentially allowing an attacker to execute arbitrary code.

Our harness focused on GIF encoding leaves two critical security gaps. It ignores non-GIF formats (JPEG, PNG, etc.) handled by `sixel_helper_load_image_file` in `loader.c`. These formats involve complex parsing logic and dependencies that remain unexercised. It also misses the Sixel-to-Pixel decoding path, such as `sixel_decode` in `fromsixel.c`. This is a critical area for terminal security, as this code parses untrusted characters sent by remote servers; a bug here could lead to shell access.

7 Binary-Only Fuzzing with QEMU Mode

We evaluated `libsixel` (v1.8.7) in a black-box scenario using AFL++ QEMU mode (`-Q`). Vanilla versions of the harness and library were built using standard `gcc/g++` with no

instrumentation or sanitizers. Performance after 30 minutes is summarized in Table 5:

1. **Execution Speed:** Instrumented mode is $\sim 5.6\times$ faster (averaging ~ 632 exec/s vs ~ 113 exec/s) due to **persistent mode** (`__AFL_LOOP`) vs. QEMU's JIT overhead and the cost of forking for each execution.
2. **Edges Discovered:** QEMU mode found $\sim 38\%$ more edges (2,826 vs 2,039) by instrumenting the entire address space, capturing paths in shared libraries (e.g., `libc`) that are black-boxes to source instrumentation.
3. **Corpus Count:** Higher throughput in the instrumented mode leads to a higher exploration rate and a larger corpus (1,332 vs 754).

Combined with QASan, we identified a heap-buffer underflow after disabling the `TRUEVISION` patch (which filters tiny inputs). QASan validates memory library calls against a shadow map, promoting "soft" corruptions to detectable crashes.

8 Instrumentation Depth and Performance

The total instrumented edge space reported by AFL++ at startup shows a target map size of 19,857 for this standalone target and **19,841 total edges for the final harness binary**.

These numbers are very close because both binaries statically link the full `libsixel` archive using `-Wl,--whole-archive -lsixel -Wl,--no-whole-archive`.

The final campaign reached 2054 edges, corresponding to a **bitmap coverage of 10.35%**. This means the fuzzer reached only a subset of the instrumented program. This is expected: the harness is GIF-focused and does not exercise unrelated paths such as PNG/JPEG loading, SIXEL decoding, terminal output behavior, and many rare error paths.

We also measured execution speed using three builds of the same harness logic: no sanitizer with fork mode, ASan with fork mode, and ASan with persistent mode. The 30-second benchmark results are shown in Table 3. ASan slows fork mode because every execution pays both the normal process startup cost and the sanitizer runtime/checking overhead. The persistent ASan build is much faster because `__AFL_LOOP` keeps the process alive and reuses the initialized library and sanitizer state across many test cases, avoiding the fork/exec and initialization cost for each input.

9 Appendix

9.1 Fuzzing Speed Measurements

Profile	Loader	Time	Execs	Speed
fast	0	74s	241,802	3235.72 exec/s
extended	0	73s	195,386	2656.29 exec/s
fast	1	74s	124,718	1670.21 exec/s
extended	1	73s	51,441	698.60 exec/s

Table 1: AFL++ speed comparison across seed profile and loader configurations.

Metric	-t 1000	-t 5000
Run time	74s	74s
Execs done	227,021	215,103
Speed	3038.45 exec/s	2878.36 exec/s
Corpus count	717	762
Bitmap coverage	9.34%	9.61%
Edges found	1853	1905
Crashes	26	29
Hangs	8	0

Table 2: Timeout A/B test using SEED_PROFILE=fast BROAD_LOADER=0.

9.2 Instrumentation Speed Benchmark

Configuration	Run time	Speed
No sanitizer + fork mode	30s	44.08 exec/s
ASan + fork mode	30s	26.33 exec/s
ASan + persistent mode	30s	2848.27 exec/s

Table 3: Execution-speed benchmark for the same harness logic under three instrumentation/runtime configurations.

9.3 Fuzzing Strategy Yields

Strategy	Yield	Impact
Bit flips	29/40.3k, 20/40.3k, 16/40.2k	Small
Byte flips	2/5035, 2/5025, 4/5005	Small
Arithmetic	86/351k, 31/696k, 27/693k	Moderate
Known ints	3/45.1k, 5/189k, 16/279k	Small
Dictionary	136/458k, 0/459k, 0/0, 0/0	Useful
Havoc/splice	973/1.88M, 0/0	Dominant
Py/custom/rq	unused, unused, unused, unused	Not used
Trim/eff	8.57%/481k, 98.83%	Corpus cleanup

Table 4: AFL++ strategy-yield summary. Havoc produced the most interesting inputs, while dictionary mutations also contributed meaningful new paths.

9.4 Campaign Analysis

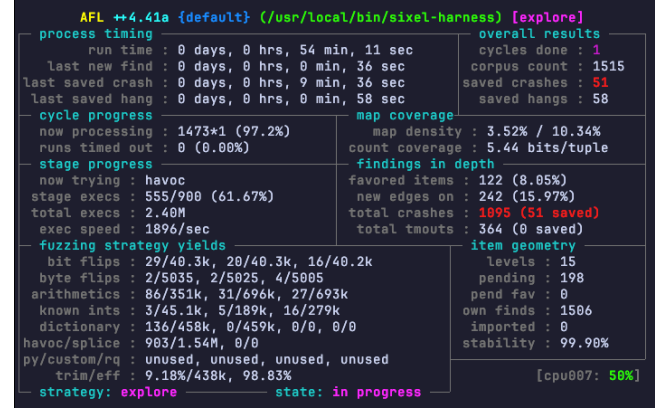


Figure 1: AFL++ status screen during the greybox campaign.

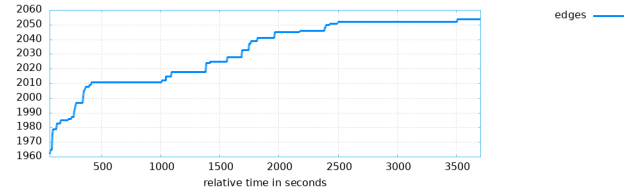


Figure 2: AFL++ edge coverage over time.

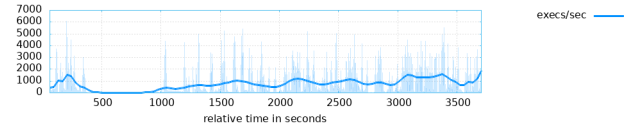


Figure 3: AFL++ execution speed over time.

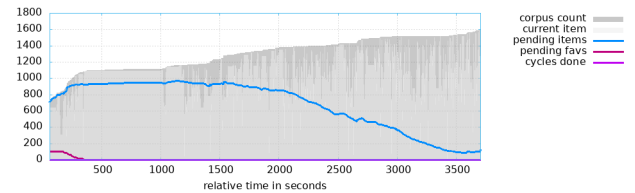


Figure 4: High-frequency AFL++ plot metrics.

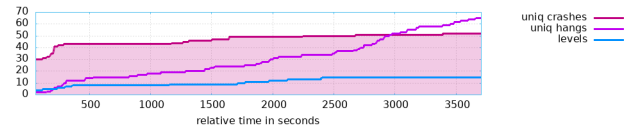


Figure 5: Low-frequency AFL++ plot metrics.

9.5 Crash Triage

```

1  ERROR: AddressSanitizer: heap-buffer-overflow
2  WRITE of size 1
3  #0 0x... in gif_init_frame /src/libsixel/src/
   fromgif.c:241:61
4  #1 0x... in load_gif /src/libsixel/src/fromgif.c:675:22
5  #2 0x... in load_with_builtin /src/libsixel/src/
   loader.c:948:18
6  #3 0x... in sixel_helper_load_image_file /src/libsixel/src/
   loader.c:1462:18
7  #4 0x... in sixel_encoder_encode /src/libsixel/src/
   encoder.c:1816:14
8  #5 0x... in main /src/harness.c:215:9
9
10 SUMMARY: AddressSanitizer: heap-buffer-overflow /src/
   libsixel/src/fromgif.c:241:61 in gif_init_frame

```

Listing 1: Backtrace of crash 47.

9.5.1 AFL-tmin

```

1  mkdir -p triage
2
3  docker run --rm \
4  -v "$PWD:/host" \
5  -e ASAN_OPTIONS=detect_leaks=0:abort_on_error=1:symbolize=1
   \
6  libsixel-fuzzer \
7  afl-tmin -m none -t 1000 \
8  -i '/host/outputs/20260514-142308/default/crashes/
   id:000047...' \
9  -o /host/triage/id000047.min \
10 -- /usr/local/bin/sixel-harness

```

Listing 2: Command to minimize the seed.

```

1  Read 51 bytes from input.
2  Program exits with a signal, minimizing in crash mode.
3  File size reduced by: 23.53% (to 39 bytes)
4  Characters simplified: 69.23%
5  Number of execs done: 249
6  Output written to: triage/id000047.min

```

Listing 3: AFL-tmin result.

```

1  docker run --rm \
2  -v "$PWD:/host" \
3  -e ASAN_OPTIONS=detect_leaks=0:abort_on_error=1:symbolize=1 \
4  libsixel-fuzzer \
5  /bin/bash -lc '/usr/local/bin/sixel-harness < /host/triage/
   id000047.min'

```

Listing 4: Reproducing the crash with minimized seed.

```

1  docker run --rm \
2  -v "$PWD:/host" \
3  -e ASAN_OPTIONS=detect_leaks=0:abort_on_error=1:symbolize=1
   \
4  libsixel-fuzzer \
5  /bin/bash -lc 'img2sixel -o /dev/null -g -p 256 -q auto -d
   auto -t rgb -f auto -s auto -E auto -B "#FF0000" -l auto /
   host/triage/id000047.min.gif'

```

Listing 5: Running img2sixel with minimized input

9.6 QEMU Performance Comparison

Axis	Instrumented	QEMU mode
Exec speed	632 exec/s	113 exec/s
Edges discovered	2,039	2,826
Corpus count	1,332	754

Table 5: Performance comparison after 30 minutes (0.5h) of wall-clock time.

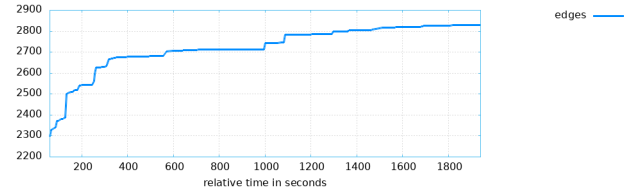


Figure 6: AFL++ QEMU mode edge coverage over time.

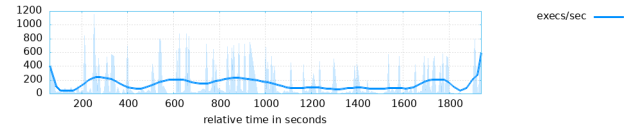


Figure 7: AFL++ QEMU mode execution speed over time.